

Annexure A
Coding Challenge

Question 1: Bayesian Network for Disease Prediction

A healthcare organization wants to predict whether a patient has Diabetes based on the symptoms Obesity and High Blood Sugar.

Coding Challenge:

Write a Python program to:

- Construct a Bayesian Network using the given variables.
- Define suitable Conditional Probability Tables (CPTs).
- Perform probabilistic inference to predict the probability of Diabetes when both symptoms are present.
- Display the predicted probability.

Question 2: Hidden Markov Model for Weather Prediction

A weather forecasting agency records daily observations as Sunny, Cloudy, and Rainy. The actual weather conditions are hidden and need to be predicted.

Coding Challenge:

Write a Python program to:

- Implement a Hidden Markov Model (HMM).
- Define transition and emission probabilities.
- Predict the hidden weather states for a given sequence of observations.
- Print the predicted sequence of hidden states.

Question 3: Markov Random Field for Image Denoising

A computer vision company wants to remove noise from grayscale images using a Markov Random Field (MRF).

Coding Challenge:

Write a Python program to:

- Represent a small image as a graph where each pixel is a node.
- Model neighboring pixel relationships using an undirected graph.
- Simulate one iteration of image smoothing based on neighboring pixel values.
- Display the original and updated pixel values.

Question 4: Conditional Random Field for Named Entity Recognition

A customer support system needs to identify Customer Names, Order IDs, and Product Names from support emails.

Coding Challenge:

Write a Python program to:

- Create sequential training data with word-level features.
- Train a Conditional Random Field (CRF) model.
- Predict entity labels for a new sentence.
- Display the predicted labels.

Question 5: Bayesian Network Learning from Data

An insurance company wants to learn probabilistic relationships among Age, Income, Vehicle Type, and Insurance Claim using historical customer data.

Coding Challenge:

Write a Python program to:

- Load a dataset using Pandas.
- Learn the Bayesian Network structure from the data.
- Perform inference to predict the probability of an insurance claim for a new customer.
- Display the learned network structure and prediction results.

Question 1: Bayesian Network for Disease Prediction

Python Program

```
# Install pgmpy if required
```

```
# !pip install pgmpy
```

```
from pgmpy.models import DiscreteBayesianNetwork
```

```
from pgmpy.factors.discrete import TabularCPD
```

```
from pgmpy.inference import VariableElimination
```

```
# Create Bayesian Network
```

```
model = DiscreteBayesianNetwork([
```

```
    ('Obesity', 'Diabetes'),
```

```
    ('HighBloodSugar', 'Diabetes')
```

```
])
```

```
# CPT for Obesity
```

```
cpd_obesity = TabularCPD(
```

```
    variable='Obesity',
```

```
    variable_card=2,
```

```
    values=[[0.7], [0.3]],
```

```
    state_names={'Obesity': ['No', 'Yes']}
```

```
)
```

```
# CPT for High Blood Sugar
```

```
cpd_sugar = TabularCPD(
```

```
    variable='HighBloodSugar',
```

```
    variable_card=2,
```

```

        values=[[0.6], [0.4]],
        state_names={'HighBloodSugar': ['No', 'Yes']}
    )

# CPT for Diabetes
cpd_diabetes = TabularCPD(
    variable='Diabetes',
    variable_card=2,
    values=[
        [0.99, 0.80, 0.85, 0.20], # No
        [0.01, 0.20, 0.15, 0.80] # Yes
    ],
    evidence=['Obesity', 'HighBloodSugar'],
    evidence_card=[2, 2],
    state_names={
        'Diabetes': ['No', 'Yes'],
        'Obesity': ['No', 'Yes'],
        'HighBloodSugar': ['No', 'Yes']
    }
)

# Add CPTs to model
model.add_cpds(cpd_obesity, cpd_sugar, cpd_diabetes)

# Check model
print("Model valid:", model.check_model())

# Perform inference
inference = VariableElimination(model)

```

```
result = inference.query(  
    variables=['Diabetes'],  
    evidence={  
        'Obesity': 'Yes',  
        'HighBloodSugar': 'Yes'  
    }  
)
```

```
print("\nProbability of Diabetes:")  
print(result)
```

Expected result

For the given CPT:

Diabetes = No → 0.20

Diabetes = Yes → 0.80

Therefore:

Predicted probability of Diabetes = 80%

Question 2: Hidden Markov Model for Weather Prediction

Here, the **actual weather is hidden**, while observations are visible.

Python Program

```
# !pip install hmmlearn
```

```
import numpy as np
```

```
from hmmlearn import hmm
```

```
# Create HMM model
```

```
model = hmm.CategoricalHMM(n_components=3, random_state=42)
```

```
# Hidden states:
```

```
# 0 = Sunny
```

```
# 1 = Cloudy
```

```
# 2 = Rainy
```

```
# Initial probabilities
```

```
model.startprob_ = np.array([
```

```
    0.5, # Sunny
```

```
    0.3, # Cloudy
```

```
    0.2 # Rainy
```

```
])
```

```
# Transition probabilities
```

```
model.transmat_ = np.array([
```

```
    [0.6, 0.3, 0.1], # Sunny -> Sunny, Cloudy, Rainy
```

```
    [0.3, 0.4, 0.3], # Cloudy
```

```
    [0.1, 0.3, 0.6] # Rainy
```

```
])
```

```
# Emission probabilities
```

```
# Observations:
```

```
# 0 = Dry
```

```
# 1 = Humid
```

```
# 2 = Wet
```

```
model.emissionprob_ = np.array([
```

```
    [0.7, 0.2, 0.1], # Sunny
```

```
    [0.3, 0.5, 0.2], # Cloudy
```

```
    [0.1, 0.3, 0.6] # Rainy
```

```
])
```

```

# Observation sequence
observations = np.array([
    [0],
    [1],
    [2],
    [2],
    [1]
])

# Predict hidden states
log_probability, hidden_states = model.decode(
    observations,
    algorithm="viterbi"
)

weather = ["Sunny", "Cloudy", "Rainy"]

print("Predicted hidden weather states:")

for state in hidden_states:
    print(weather[state])

```

Output

The program prints something like:

Predicted hidden weather states:

Sunny

Cloudy

Rainy

Rainy

Cloudy

Note: The exact sequence depends on the transition and emission probabilities.

Question 3: Markov Random Field for Image Denoising

A small grayscale image is represented as a matrix. Each pixel is connected to its neighboring pixels.

Python Program

```
import numpy as np

# Noisy grayscale image
image = np.array([
    [50, 52, 200, 51, 50],
    [49, 55, 210, 53, 52],
    [48, 50, 205, 54, 51],
    [47, 49, 200, 52, 50],
    [46, 48, 195, 50, 49]
], dtype=float)

print("Original Noisy Image:")
print(image)

# Create updated image
updated_image = image.copy()

rows, cols = image.shape

# One iteration of smoothing
for i in range(rows):
    for j in range(cols):
```



```

neighbors = []

# Top
if i > 0:
    neighbors.append(image[i-1, j])

# Bottom
if i < rows - 1:
    neighbors.append(image[i+1, j])

# Left
if j > 0:
    neighbors.append(image[i, j-1])

# Right
if j < cols - 1:
    neighbors.append(image[i, j+1])

# Average neighboring pixels
updated_image[i, j] = np.mean(neighbors)

print("\nUpdated Image after MRF Smoothing:")
print(np.round(updated_image, 2))

```

Logic

For every pixel:

1. Find neighboring pixels.
2. Calculate their average.
3. Replace the pixel with the average.

4. This reduces sudden noise.

Example:

Original Pixel

↓

Check neighbors

↓

Calculate average

↓

Update pixel

Question 4: Conditional Random Field for Named Entity Recognition

For this question, we can use the sklearn-crfsuite library.

Python Program

```
# Install library if required
```

```
# !pip install sklearn-crfsuite
```

```
import sklearn_crfsuite
```

```
# Training sentences
```

```
X_train = [  
    [  
        {"word": "John", "is_capitalized": True},  
        {"word": "ordered", "is_capitalized": False},  
        {"word": "Laptop", "is_capitalized": True},  
        {"word": "12345", "is_digit": True}  
    ],  
    [  
        {"word": "Alice", "is_capitalized": True},  
        {"word": "bought", "is_capitalized": False},
```

```

        {"word": "Phone", "is_capitalized": True},
        {"word": "67890", "is_digit": True}
    ],
    [
        {"word": "Rahul", "is_capitalized": True},
        {"word": "wants", "is_capitalized": False},
        {"word": "Headphones", "is_capitalized": True},
        {"word": "54321", "is_digit": True}
    ]
]

# Labels
# B-CUSTOMER = beginning of customer name
# B-PRODUCT = product name
# B-ORDER = order ID
y_train = [
    ["B-CUSTOMER", "O", "B-PRODUCT", "B-ORDER"],
    ["B-CUSTOMER", "O", "B-PRODUCT", "B-ORDER"],
    ["B-CUSTOMER", "O", "B-PRODUCT", "B-ORDER"]
]

# Train CRF
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    max_iterations=100
)

crf.fit(X_train, y_train)

```

```
# New sentence
X_test = [
    {"word": "David", "is_capitalized": True},
    {"word": "ordered", "is_capitalized": False},
    {"word": "Tablet", "is_capitalized": True},
    {"word": "98765", "is_digit": True}
]
```

```
# Prediction
prediction = crf.predict(X_test)
```

```
print("Predicted Labels:")
```

```
for word, label in zip(
    ["David", "ordered", "Tablet", "98765"],
    prediction[0]
):
    print(word, "->", label)
```

Expected output

Predicted Labels:

David -> B-CUSTOMER

ordered -> O

Tablet -> B-PRODUCT

98765 -> B-ORDER

Meaning

Label	Meaning
-------	---------

B-CUSTOMER	Customer Name
------------	---------------

Label	Meaning
B-PRODUCT	Product Name
B-ORDER	Order ID
O	Not an entity

Question 5: Bayesian Network Learning from Data

This program creates a small dataset using Pandas, learns the Bayesian Network structure, and performs inference.

Python Program

```
# !pip install pgmpy

import pandas as pd

from pgmpy.estimators import HillClimbSearch, BayesianEstimator
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.inference import VariableElimination

# Create dataset
data = pd.DataFrame({
    'Age': [
        'Young', 'Young', 'Young', 'Middle',
        'Middle', 'Middle', 'Old', 'Old',
        'Old', 'Young', 'Middle', 'Old'
    ],
    'Income': [
        'Low', 'High', 'Low', 'High',
        'Medium', 'Low', 'High', 'Medium',
        'Low', 'Medium', 'High', 'Medium'
    ]
})
```

```
],
```

```
'Vehicle': [
```

```
    'Car', 'Bike', 'Car', 'Car',
```

```
    'Bike', 'Bike', 'Car', 'Car',
```

```
    'Bike', 'Bike', 'Car', 'Bike'
```

```
],
```

```
'Claim': [
```

```
    'No', 'Yes', 'No', 'Yes',
```

```
    'No', 'Yes', 'No', 'Yes',
```

```
    'Yes', 'No', 'No', 'Yes'
```

```
]
```

```
})
```

```
print("Dataset:")
```

```
print(data)
```

```
# Learn network structure
```

```
hc = HillClimbSearch(data)
```

```
best_model = hc.estimate()
```

```
print("\nLearned Network Structure:")
```

```
print(list(best_model.edges()))
```

```
# Create Bayesian Network using learned edges
```

```
model = DiscreteBayesianNetwork(best_model.edges())
```

```

# Learn CPDs
model.fit(
    data,
    estimator=BayesianEstimator,
    prior_type="BDeu"
)

# Check model
print("\nModel Valid:", model.check_model())

# Perform inference
inference = VariableElimination(model)

# Predict probability of claim
result = inference.query(
    variables=['Claim'],
    evidence={
        'Age': 'Young',
        'Income': 'Low',
        'Vehicle': 'Car'
    }
)

print("\nProbability of Insurance Claim:")
print(result)

```

Expected type of output

Dataset:

	Age	Income	Vehicle	Claim
0	Young	Low	Car	No

1 Young High Bike Yes

...

Learned Network Structure:

[('Age', 'Claim'), ('Income', 'Claim'), ...]

Probability of Insurance Claim:

+-----+-----+

| Claim | phi(Claim) |

+=====+

| No | ... |

| Yes | ... |

+-----+-----+